

Agent Team Prompts

7 Copy-Paste Prompts for Claude Code Agent Teams

- 01 Content Repurposing Pipeline
- 02 Pitch Deck Builder (with Task Dependencies)
- 03 RFP Response Generator
- 04 Competitive Intelligence Report
- 05 Advisory Board Decision Analysis
- 06 Full Marketing Campaign Launch
- 07 Build a Personal AI Business Assistant

#01

Content Repurposing Pipeline

Use case: Repurpose a single video transcript into 4 platform-specific content pieces with zero overlap.

Create an agent team to repurpose a video transcript into content for 4 platforms.

The transcript is at `/path/to/your/transcript.md`.

Spawn 4 teammates:

1. Blog Writer – Transform the transcript into a 1,200-word blog post. Structure: hook paragraph, 3-5 subheadings with key insights, conclusion with CTA. Write in first person. Save to `outputs/agent_teams_demo/content_repurposing/blog_post.md`.

2. LinkedIn Writer – Write a LinkedIn post (150-250 words). Use a hook line, 3 key takeaways as short paragraphs, and end with a question to drive comments. No hashtag spam – max 3 relevant hashtags. Save to `outputs/agent_teams_demo/content_repurposing/linkedin_post.md`.

3. Newsletter Writer – Write an email newsletter edition (300-500 words). Subject line + preview text + body. Conversational tone, one clear CTA. Save to `outputs/agent_teams_demo/content_repurposing/newsletter.md`.

4. Twitter Thread Writer – Write a 7-10 tweet thread. Tweet 1 is the hook. Each tweet is self-contained but builds on the last. Final tweet is a CTA. Save to `outputs/agent_teams_demo/content_repurposing/twitter_thread.md`.

Before writing, each teammate should read the full transcript and identify the 3 most compelling insights. Have them share their chosen insights with each other to ensure no two platforms lead with the same angle – each piece should feel fresh, not repetitive.

After all 4 are done, synthesize a summary comparing the angles each teammate chose and flag any messaging inconsistencies.

#02

Pitch Deck Builder (with Task Dependencies)

Use case: Research-backed pitch deck with sequential handoffs — Researcher → Writer → Designer.

Create an agent team to build a 12-slide pitch deck about how AI automation is transforming small business operations in 2026.

Spawn 3 teammates with task dependencies:

1. Researcher – Find 8-10 key data points, stats, and supporting evidence about AI automation adoption among small businesses. For each data point, include the source and a one-sentence explanation of why it matters. Save findings to `outputs/agent_teams_demo/pitch_deck/research_brief.md`. This task must complete before the Writer begins.

2. Slide Writer – Using the Researcher's brief, write the content for a 12-slide deck:
Slide 1: Title + subtitle
Slide 2: The problem / opportunity
Slides 3-4: Key data points (pick the 4 strongest from research)
Slides 5-8: Core argument / solution (one point per slide)
Slide 9: Case study or example
Slide 10: Objection handling
Slide 11: Summary / key takeaway
Slide 12: CTA / next steps
Each slide: headline (max 8 words), 3-4 bullet points (max 12 words each), and speaker notes (2-3 sentences). Save to `outputs/agent_teams_demo/pitch_deck/slide_content.md`. This task must complete before the Designer begins.

3. Designer – Using the Slide Writer's content, build the actual .pptx file using Python (python-pptx library). Use a clean, professional design: dark background (#1a1a2e), white text, accent color (#E07A4F). Include proper slide layouts, consistent fonts (Arial or Helvetica), and logical visual hierarchy. Save to `outputs/agent_teams_demo/pitch_deck/pitch_deck.pptx`.

Require plan approval for the Designer before they start building – I want to review the design approach first.

#03

RFP Response Generator

Use case: Parallel proposal writing with cross-referenced requirements and consistency checks.

Create an agent team to respond to a Request for Proposal.

The RFP document is here:

<https://canadabuys.canada.ca/en/tender-opportunities/tender-notice/bc006-2026>

Our company: We are a 15-person AI consulting firm specializing in building custom automation workflows for mid-market companies. We use tools like Claude Code, n8n, and Supabase. Our average project size is \$25K-\$75K and we've completed 40+ projects in the last 18 months. Our differentiator is that we build systems clients can maintain themselves after handoff.

Spawn 4 teammates:

1. RFP Analyst – Read the full RFP and extract:

All explicit requirements (numbered list)

Evaluation criteria and their weights

Submission requirements (format, deadline, page limits)

Key questions or ambiguities that need clarification

Save analysis to `outputs/agent_teams_demo/rfp_response/rfp_analysis.md`. This must complete before Section Writers begin.

2. Company Capability Researcher – Read our company information above and build a capability matrix that maps our strengths to common RFP categories: technical approach, team qualifications, past performance, management approach, pricing strategy. Identify 3-5 differentiators. Save to `outputs/agent_teams_demo/rfp_response/capability_matrix.md`. This must complete before Section Writers begin.

3. Section Writer A – Using the RFP analysis and capability matrix, write:

Executive Summary (1 page)

Technical Approach section

Management Approach section

Save to `outputs/agent_teams_demo/rfp_response/proposal_sections_a.md`.

4. Section Writer B – Using the RFP analysis and capability matrix, write:

Team Qualifications section (with placeholder bios)

Past Performance section (with placeholder case studies)

Pricing Narrative (not actual numbers – the strategic framing)

Save to `outputs/agent_teams_demo/rfp_response/proposal_sections_b.md`.

After both Section Writers finish, review all sections for:

Consistent tone and terminology

No contradictions between sections

Every RFP requirement addressed (cross-reference against the Analyst's checklist)

Flag any requirements we didn't address

Save final assembled proposal to
outputs/agent_teams_demo/rfp_response/full_proposal.md.

#04

Competitive Intelligence Report

Use case: Parallel competitor analysis with shared findings and synthesized battlecard.

Create an agent team to build a competitive intelligence report.

Target product: Claude Code – the AI-powered coding assistant CLI from Anthropic.

Competitors to analyze: AntiGravity, Cursor, Codex, GitHub Copilot

Spawn 4 teammates

Document the following for each platform (latest/greatest info as of 2026):

- >Product overview and core features
- >Pricing model and tiers
- >Target audience and positioning
- >Strengths (what they do better than Claude Code)
- >Weaknesses (where Claude Code has an advantage)
- >Recent news, funding, or product launches
- >Save to outputs/agent_teams_demo/competitive_intel/[competitor]_analysis.md.

Have each analyst share their top 3 findings with the group before the Synthesis Lead begins, so the Lead has context on what surprised each analyst most.

#05

Advisory Board Decision Analysis

Use case: 5 agents debate a high-stakes business decision from different angles, then converge on a recommendation.

Create an agent team to analyze a complex business decision.

The question: "Should we launch a \$7,500 live 6-week AI Leadership Bootcamp for executives and CEOs who want to manage AI teams and integrate AI into their operations – or should we start with a \$2,000 self-paced course first to validate demand?"

Context: We run Prompt Advisers (<https://www.promptadvisers.com/>), an AI consulting and education company. We already have a free Skool community with 800+ members. Our audience is a mix of solopreneurs, agency owners, and mid-level managers – but we want to go upmarket to C-suite executives. The bootcamp would cover: building an AI strategy, managing AI-augmented teams, evaluating AI tools for enterprise, and measuring ROI on AI initiatives.

Spawn 5 teammates to investigate different angles. Have them talk to each other to challenge each other's assumptions, like an advisory board debate.

1. Market Researcher – Analyze the executive AI education market: who's selling high-ticket AI programs to C-suite, what they charge, what formats work (live cohort vs self-paced vs hybrid), typical completion rates, and what executives actually want to learn. Quantify the total addressable market. Save initial findings to `outputs/agent_teams_demo/big_problem/market_research.md`.

2. Audience Gap Analyst – Investigate the gap between our current audience (solopreneurs, agency owners) and the target audience (executives, CEOs). What marketing channels reach C-suite? What credibility signals do they need before spending \$7,500? How different is the sales cycle? What would we need to change about our brand positioning? Save to `outputs/agent_teams_demo/big_problem/audience_analysis.md`.

3. Financial Modeler – Build a rough financial model comparing both scenarios: Scenario A (\$7,500 bootcamp, 15 seats per cohort, live delivery) vs Scenario B (\$2,000 self-paced, unlimited seats, async). For each: estimated build costs, revenue per cohort, profit margin, break-even point, time to first dollar, and scaling trajectory over 12 months. Save to `outputs/agent_teams_demo/big_problem/financial_model.md`.

4. Competitive Strategist – Map who's already selling executive AI education: Maven courses, Wharton/Harvard executive programs, Reforge, consulting firms offering workshops. Their pricing, positioning, cohort sizes, and how they'd likely view us as a new entrant. Identify if there's a positioning gap we can exploit that universities and big firms can't fill. Save to `outputs/agent_teams_demo/big_problem/competitive_strategy.md`.

5. Devil's Advocate – Your job is to find reasons this could fail. Challenge every other teammate's assumptions. Can a YouTube creator credibly sell to Fortune 500 execs?

Is the \$7,500 price point realistic without enterprise sales infrastructure? What if executives don't take an online bootcamp seriously? What if we alienate our existing community by going upmarket? Find the risks nobody wants to talk about. Save your contrarian analysis to `outputs/agent_teams_demo/big_problem/devils_advocate.md`.

After each teammate completes their initial analysis, have them share their top finding with the group. The Devil's Advocate should then challenge at least 2 other teammates' conclusions. Let the debate play out.

Once consensus (or informed disagreement) emerges, synthesize into a single executive brief with:

GO / NO-GO / CONDITIONAL recommendation

Top 3 reasons for the recommendation

Top 3 risks regardless of decision

Suggested next steps

Save to `outputs/agent_teams_demo/big_problem/executive_brief.md`.

#06

Full Marketing Campaign Launch

Use case: Coordinated campaign across email, social, ads, and landing page — with cross-channel consistency review.

Create an agent team to build a complete marketing campaign for a product launch.

The product: A premium noise-canceling headphone called "FocusPods Pro" priced at \$249, targeting remote workers and creators who need deep focus in noisy environments. Key selling points: 40-hour battery life, spatial audio for virtual meetings, a "Focus Mode" that filters out human voices but lets through doorbells and alarms, and a built-in Pomodoro timer with haptic taps. Launching on our DTC website with a 30-day money-back guarantee.

Spawn 4 teammates:

1. Email Marketer – Write a 3-email launch sequence:

Email 1 (teaser, 2 days before): Build anticipation, hint at what's coming, subject line that gets opens

Email 2 (launch day): Announce FocusPods Pro, 3 key benefits, clear buy-now CTA

Email 3 (follow-up, 2 days after): Social proof angle, early buyer reactions, second chance CTA with a "first 100 buyers get a free carrying case" urgency play

Each email: subject line, preview text, body (150-250 words), CTA button text. Save to outputs/agent_teams_demo/marketing_campaign/email_sequence.md.

2. Social Media Manager – Create platform-specific launch content:

LinkedIn post (150-250 words, professional angle: productivity and remote work)

Twitter/X thread (7 tweets, hook first, CTA last)

Instagram caption (lifestyle angle, short and punchy, hashtag strategy)

TikTok script (30-second hook: "POV: your coworker won't stop talking during your deep work block")

Save to outputs/agent_teams_demo/marketing_campaign/social_content.md.

3. Ad Copywriter – Write 3 variations of ad copy for paid promotion:

Variation A: Problem-agitation-solution (noisy environment, can't focus, FocusPods fix it)

Variation B: Social proof / results ("87% of beta testers reported deeper focus in week 1")

Variation C: Us-vs-them comparison (AirPods Max vs FocusPods Pro feature table framing)

Each variation: headline (max 40 chars), body (max 125 chars), CTA text. Include targeting suggestions (audience, interests, platforms). Save to outputs/agent_teams_demo/marketing_campaign/ad_copy.md.

4. Landing Page Writer – Write the full copy for a product landing page:

Hero section (headline + subheadline + CTA + "As seen in TechCrunch, Wired, The Verge" placeholder)

Product feature blocks (Focus Mode, 40hr battery, Spatial Audio, Pomodoro Timer – each with icon placeholder + headline + 2 sentences)

Social proof section (3 placeholder testimonials from remote workers, a creator, and a startup CEO)

Comparison table (FocusPods Pro vs AirPods Max vs Sony WH-1000XM6)

FAQ section (5 common objections: price, comfort, compatibility, return policy, delivery)

Final CTA section with urgency

Save to outputs/agent_teams_demo/marketing_campaign/landing_page_copy.md.

After all 4 finish, review all content for:

Consistent messaging and tone across all channels

No contradictory claims between email, social, ads, and landing page

The same core value proposition appears in every piece

CTA links all point to the same destination

Save a campaign summary with messaging guidelines to

outputs/agent_teams_demo/marketing_campaign/campaign_summary.md.

#07

Build a Personal AI Business Assistant

Use case: Subagent analysis → Agent team build. A working Telegram bot with skill routing, memory, and business context.

Create an agent team to build a personal AI business assistant – a better, customized version of OpenClaw built specifically to run my company, Prompt Advisers (<https://www.promptadvisers.com/>).

The end goal: a working CLI I can start with a single command that connects to Telegram, understands my business context, picks the right tool for any task, and helps me run my company day-to-day.

First, before spawning the team, use a subagent to clone and analyze the real OpenClaw repo. This shows the difference between a subagent (one agent reporting back) and an agent team (multiple agents collaborating):

Step 1 (Subagent): Clone <https://github.com/anthropics/openclawai> and read through the codebase. Summarize:

The overall architecture and how components connect

Key design patterns used

What technologies and dependencies it relies on

The 3 best ideas we should steal for our version

The 3 things we should do differently (simpler, more focused)

Save the analysis to `outputs/agent_teams_demo/pa_assistant/repo_analysis.md`.

Step 2 (Agent Team): Now, using insights from the subagent's analysis, spawn 5 teammates to build our custom version. Require plan approval from each before they start coding.

1. Architect – Design the system architecture for "PA Assistant" (Prompt Advisers Assistant):

Component diagram showing how all pieces connect

API contracts between components

Tech stack: Python, `python-telegram-bot` for Telegram integration, SQLite for memory, Claude API for reasoning

A skill router that picks the best skill for any incoming request

File structure and module boundaries

What we're borrowing from OpenClaw vs. what we're building custom

Save to `outputs/agent_teams_demo/pa_assistant/architecture.md`. This must complete and be approved before other teammates begin.

2. Telegram Interface – Build the Telegram bot layer:

Bot setup with `python-telegram-bot` library

Message handler that receives text, voice notes (transcribed), images, and documents

Response formatting (markdown, inline keyboards for confirmations)

Conversation thread tracking (group vs DM context)

Startup command that registers the bot and starts polling

Own all files in outputs/agent_teams_demo/pa_assistant/src/telegram_bot/. Run tests after implementation.

3. Skill Router and Tools – Build the intelligence layer that picks the right action:

A skill registry with at least 8 skills relevant to running Prompt Advisers:

"draft-email" – draft client emails, follow-ups, outreach

"content-repurpose" – turn a transcript or blog post into social posts

"meeting-prep" – summarize context before a consulting call

"proposal-writer" – draft project proposals and SOWs

"schedule-check" – query Google Calendar availability

"community-pulse" – summarize recent Skool community activity

"invoice-reminder" – draft payment follow-up messages

"research" – web search and summarize findings on a topic

Intent detection that routes incoming messages to the right skill

Fallback: if no skill matches, use Claude as a general assistant

Own all files in outputs/agent_teams_demo/pa_assistant/src/skills/. Run tests after implementation.

4. Memory and Context – Build the business context system:

Short-term memory (current conversation thread)

Long-term memory (client names, project history, preferences, frequently used commands)

Business context file that the assistant loads on startup (company info, services, pricing, team members)

SQLite storage with retrieval API

A "learn" command: when I say "remember that Client X prefers async updates", it stores it

Own all files in outputs/agent_teams_demo/pa_assistant/src/memory/. Run tests after implementation.

5. Integration and CLI – Wire everything together and make it runnable:

A single entry point: python main.py that starts the Telegram bot and loads all skills

A setup script: setup.sh that installs dependencies, creates the SQLite database, and prompts for API keys (Telegram bot token, Claude API key)

Environment variable handling via .env file

Health check: on startup, verify all connections (Telegram, Claude API, database) and report status

A README.md with clear setup instructions: how to create a Telegram bot via BotFather, get the token, configure .env, and run

Wire all components together based on the Architect's contracts

Own all files in outputs/agent_teams_demo/pa_assistant/src/main.py, setup.sh, and README.md. Run integration tests after wiring.

After all teammates finish:

Run the full test suite

Print the exact commands to start the assistant:

```
./setup.sh
```

```
python src/main.py
```

Do a live test: send the Telegram bot a message like "draft a follow-up email to a client who hasn't responded in 5 days about their AI automation project" and verify it routes to the right skill and produces a good response

Document the result in `outputs/agent_teams_demo/pa_assistant/test_results.md`
